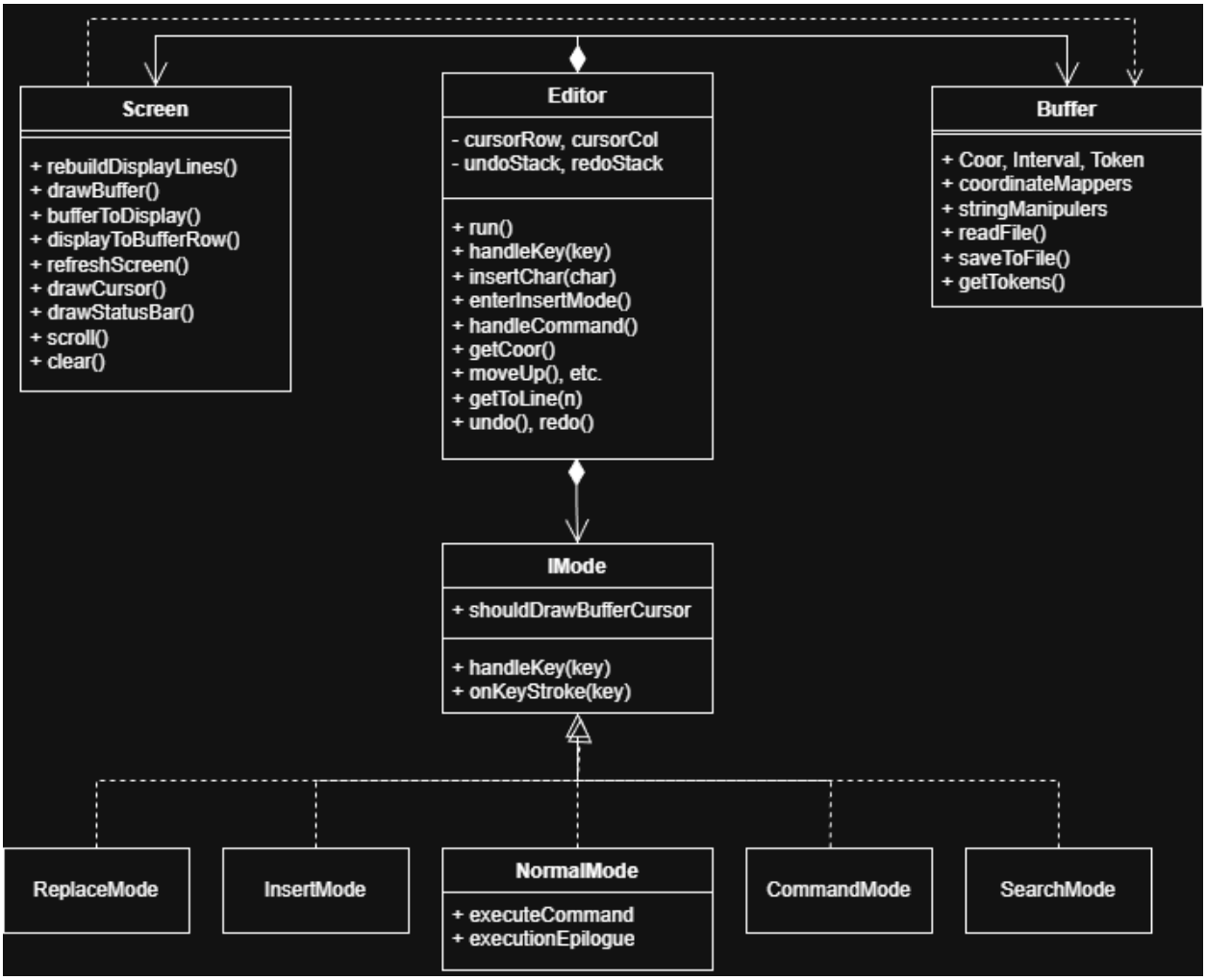
**Overview:**

Commands are viewed as multiplier1 + operator + multplier2 + motion. Motion transform current coordinate, operator operate on the sorted [cursor, transformed). Special cases such as dj are treated differently before the general execution

Some designs are different from vim. For example, when recording a macro of 2cc, and plays it twice. We would expect it to change 4 lines, but vim only changes 2 lines. In vm, the commands are interpreted as operate on the range of applying motion multiplier2 times, repeat this multiplier1 times.




**Design:**

- Buffer class
    - keeps a vector of strings, each string is one line of text. Together they become the content of the editing file

    - has a library of helper methods that takes a coordinate or an interval and return a coordinate or interval of desire, enables functional-style transformations: Coor → Coor
        - nextWord
        - firstNonWhitespaceChar
    - has helper methods that take actions on the buffer
        - del
        - insert
        - replace
    - implemented with a compiler getTokens
        - parse the buffer into tokens sequentially
        - implements a finite state machine
            - starts in normal mode
            - if a " is encountered, enter string mode, exits when another " is met
            - if a // or /* is entered, enter comment mode, exit at the end of the line or */ is met
            - handles escape sequence
- Screen class
    - Takes raw buffer strings or tokens and convert to displayable lines
    - Divide lines that are too long by the width of the screen to enable line wrap
    - Token types are also maintained when dividing
    - Syntax highlights according to the token type
    - Draws status bar
    - Has the option to draw cursor
    - Scrolling
        - Maintain a firstRowDisplayed which is the first screen line that is displayed onto the screen
        - [firstRowDisplayed, lastRowDisplayed] is the bound of the display
        - If the screen cursor is out of bound, move firstRowDisplayed so that cursor is on the screen
        - Scrolling commands increments or decrements firstRowDisplayed, and move the cursor to the center of the screen if possible
- Editor class
    - Incorporates strategy pattern, state pattern, template pattern
    - Interface IMode has children InsertMode, NormalMode, CommandMode, SearchMode, ReplaceMode
    - Editor asks IMode mode to handle each key input, without knowing what mode it is in
    - Every IMode must define handleKey, along with fields like whether they want the cursor to be displayed or not

- Memento pattern
  - Each undoable command calls editor's pushUndoStack, and editor keeps it
  - Redo is implemented in the same way
- Command pattern, encapsulates operator-motion commands in Command class
- Dependency Inverstion
  - Editor depends on IMode's abstract handleKey, but not its implementation
- NormalMode
  - To support commands in the forms of [mult1][op][mult2][motion], a Command class is declared to hold the parsed result from handleKey
  - When the command is ready to be executed, handleKey calls ExecuteCommand
  - ExecuteCommand
    - Motions are pure coordinate transformation functions: f: Coor → Coor
    - Operators are actions on coordinate ranges: g: (Coor, Coor) → void
    - The system composes them: g(startCoor, f^multiplier2(currentCoor))
    - Multipliers provide repetition at different levels of abstraction
    - If there are o operators and m motions, this approach reduces om combinations to o+m combinations
  - Command Parsing State Machine:
    - Tracks parsing states
    - Handles multi-character commands: f<char>, r<char>
    - Validates command completeness before execution
  - % is implemented with a braces stack
  - . is implemented with a saved lastCommand
  - Editor keeps a global copy of search result that n/N can use, maintained even after switching modes
  - Standalone commands are executed multiple times if asked and allowed
  - Undo/Redo
    - Keeps a undo stack and a redo stack
    - Push the state of the buffer and cursor coor to undo stack for every modifying commands
    - If undo is popped, the current state is pushed onto redo
    - If a modifying command is executed, redo is cleared
  - Macro Recording and Replay
    - Keeps a map from register to vector of char input

        - Records during normal execution (commands execute while recording)
        - Macro is replayed through the same pipeline as other commands
    - Command Mode
        - Detects if the working file is read-only, forbids saving to this file if it is, but allows saving to a different file name

**Extra Features:**

- Forbids saving to a read-only file, requires checking system's permission on file's write
- No delete, implicit memory handling via RAII
- Syntax highlighting
    - Context-dependent parsing (same character means different things)
    - State must persist across lines
    - Parsed the buffer from a vector of strings into vector of vector (lines) of tokens
    - Implemented via compiler with a finite state machine
    - Had to handle edge cases
        - Multiline string, comments
        - " and ' within comments
        - Escape sequences like \"
    - Coordinate mapping, buffer coordinate has to be mapped onto a coor on the screen
    - Keyword detection, in the second pass through the buffer
    - Numbers within identifiers, int temp1
    - Brace color matching and warning, has to ignore braces inside strings and comments, implemented with a stack in the second pass
    - Screen must have the additional option of transforming buffer of tokens
    - :syntax on off toggle in NormalMode
- File indentation
    - >>, << commands
    - Ctrl+I for auto indenting the entire file, determined by the number of braces the line is nested within
    - Requires a parser to identify {}, and ignoring those in strings and comments
- Summary for design techniques used
    - State machines: Tokenization and parsing use explicit state tracking
    - Two-pass algorithms: Tokenize first, then process (brace matching, keyword detection)
    - Context-aware parsing: Track string/comment state to ignore special characters
    - Coordinate transformation: Map between buffer and display coordinate systems
    - Lazy evaluation: Syntax highlighting only when enabled

- RAII: automatic memory management throughout
- System integration: POSIX system calls for file permissions
- Feature flags: Runtime toggles for syntax highlighting

**Final Question**

Have the motion returns a pair of coordinate and interval. If motions are used as standalone commands for moving cursor, the coordinate is used. If the motions are used as defining the range for the operator, the interval is used.